

Техническое задание

Реализация клиента Ozon Performance на PHP

Эталонная Python-реализация: `ozon_performance.py`. Цель — портировать функциональность на PHP с сохранением структуры выходных данных.

1. Цель проекта

Разработать PHP-библиотеку для работы с Ozon Performance API. Библиотека загружает справочник рекламных кампаний и собирает дневную статистику (показы, клики, расход, охват, видео-метрики), плюс строит сводную таблицу-агрегат `admin_audit`. Выход каждой публичной функции — структурированный массив ассоциативных массивов, эквивалентный pandas DataFrame в Python-эталоне.

2. Общая логика работы библиотеки

Все функции статистики работают по одной схеме. Ниже — последовательность шагов для `get_campaigns_daily_stat`, `get_ads_daily_stat`, `get_video_ads_daily_stat`.

1. Получить справочник кампаний. GET `/api/client/campaign` возвращает все кампании аккаунта (88 штук). Из каждой берём `id` (привести к строке) и `advObjectType` (BANNER / VIDEO_BANNER / ...).
2. Сформировать список дат. Из параметров `date_from` / `date_to` — массив отдельных дней (YYYY-MM-DD).
3. Разбить кампании на батчи. API принимает максимум 10 кампаний за раз — делим список на группы по 10.
4. Для каждой пары (день, батч) — запросить статистику. POST `/api/client/statistics` со списком ID и одним днём. API не отдаёт данные сразу — возвращает UUID задачи.
5. Ждать готовности отчёта (polling). Каждые 7 сек опрашиваем GET `/api/client/statistics/{UUID}`. Когда `state = OK` — переходим к скачиванию.
6. Скачать и распаковать отчёт. GET `/api/client/statistics/report?UUID=...` → CSV (1 кампания) или ZIP с CSV-файлами. Распаковываем, привязываем каждый CSV к своей кампании по имени файла.
7. Распарсить CSV. CSV содержит строки уровня объявления (одна строка = одно объявление x день). `get_campaigns_daily_stat`: суммировать по (`date`, `campaign_id`). `get_ads_daily_stat` / `get_video_ads_daily_stat`: оставить как есть. `get_reach_*`: взять строку 'Всего' (campaign-level reach) или ad-level строки.
8. Собрать итоговый результат. Все строки из всех дней и батчей объединить в один массив с фиксированными колонками.

Для охват-функций (`get_reach_campaigns_daily_stat`, `get_reach_ads_daily_stat`) логика немного отличается: для каждого дня D запрос делается за период [`global_start_date`, D] (не один день), а из ответа берётся кумулятивный охват. Дневной прирост (increment) вычисляется локально как разница между соседними днями.

3. Ключевые принципы работы с Ozon Performance

3.1. Авторизация

- OAuth 2.0, `grant_type=client_credentials`. POST `/api/client/token` с Content-Type: `application/json`. GET возвращает 405 — только POST.
- Ответ: `{access_token, expires_in: 1800, token_type: Bearer}`.
- Заголовок всех запросов: `Authorization: Bearer <access_token>`.
- Токен обновлять за 60 сек до истечения (`TOKEN_REFRESH_LEEWAY_SEC = 60`).
- Учётные данные: `env CLIENT_ID / CLIENT_SECRET`. Формат `CLIENT_ID: 94252485-1777222314558@advertising.performance.ozon.ru`

3.2. Базовый URL и эндпоинты

BASE_URL = https://api-performance.ozon.ru

Метод	Путь	Назначение
POST	/api/client/token	Выпуск access-токена
GET	/api/client/campaign	Список всех кампаний аккаунта
POST	/api/client/statistics	Запрос отчёта статистики (возвращает UUID)
GET	/api/client/statistics/{UUID}	Статус готовности отчёта
GET	/api/client/statistics/report	Скачивание готового отчёта

3.3. Асинхронная схема получения отчётов (submit → poll → download)

- POST /api/client/statistics → получить UUID задачи.
- Опрашивать GET /api/client/statistics/{UUID} каждые 7 сек (макс. 40 попыток ≈ 5 мин). Поле статуса: state. Значения: NOT_STARTED / IN_PROGRESS / OK / ERROR.
- После state=OK: GET /api/client/statistics/report?UUID=... → text/csv (1 кампания) или application/zip (2+ кампаний). ZIP содержит по одному CSV на кампанию.

3.4. Лимиты API — критичны

Лимит	Значение	Как соблюдать
Кампаний в одном запросе	≤ 10	Разбивать на батчи по 10
Параллельных задач генерации	≤ 3	Не submit'ить при 3 активных UUID
Скачиваний отчётов в сутки	≤ 1000	Учитывать при больших периодах
Запросов в сутки	≤ 100 000	Общий лимит (включая токен)
Интервал между submit'ами	≥ 3 сек	Выдерживать паузу в коде

3.5. Обработка ошибок и rate-limit

- HTTP 429 — exponential backoff: стартовая пауза 10 сек, удвоение, до 5 повторов. Уважать заголовок Retry-After если присутствует.
- Пейсинг submit'ов: ≥ 3 сек между POST /api/client/statistics.
- Если активных UUID уже 3 — ждать освобождения слота перед новым submit.
- state=ERROR в polling — логировать warning, пропустить батч, продолжить цикл.
- Ошибки HTTP / таймаут polling — не прерывать общий цикл. Логировать и продолжать.

3.6. Подводные камни форматов данных

- Числа в CSV — строки с запятой как десятичным разделителем: "25833,00" → 25833.0. parseNum(\$v): str_replace(',', '.', trim(str_replace(['\xc2\xa0', ' ', ' ', ' '), \$v))) → (float).
- Даты внутри CSV: DD.MM.YYYY. В запросах: YYYY-MM-DD. Нужен конвертер parseDateStr().
- Кодировка CSV: cp1251 — декодировать: mb_convert_encoding(\$csv, 'UTF-8', 'CP1251').
- Разделитель CSV: ; (не ,).
- Строка 0 CSV — метаданные (';Рекламная кампания № ...'). Заголовок: первая строка начинающаяся с 'День;':.
- Строка 'Всего' — пропускать (кроме функций охвата — брать для reach).
- Строка 'Корректировка' — пропускать (нет валидной даты).
- campaign_id — хранить как строку: (string)\$campaign['id']. В JSON поле id — integer.
- Envelope: ответ GET /api/client/campaign приходит как {"list": [...]}.

3.7. Особый случай — кумулятивная метрика «reach» (охват)

reach — количество уникальных пользователей за период. Нельзя суммировать по дням — за это отвечает API через groupBy=NO_GROUP_BY и кумулятивный запрос [global_start_date, D].

Для каждого дня D: POST /api/client/statistics с dateFrom=global_start_date, dateTo=D, groupBy=NO_GROUP_BY. Для campaign-level (4.4) — строка 'Всего' из CSV → колонка 'Охват' (API-дедуплицированный reach). Для ad-level (4.5) — ad-level строки CSV (одна на платформу), сумма 'Охват' по платформам в рамках одного объявления — допустимое упрощение, точной дедупликации (объявление × платформа) API не даёт.

increment[D] = reach[D] - reach[D-1]; для первого дня = reach (fillna).

3.8. TLS / проверка сертификатов сервера

С 2026 Ozon меняет корневой удостоверяющий центр (УЦ) TLS-сертификатов своих серверов: GlobalSign -> Harica. Плановое обновление затрагивает все публичные API Ozon (Performance API, Seller API и др.).

- Доверять актуальному СА-бандлу, а не «жёстко зашитому» (hardcoded) списку корней. Бандл должен содержать корень Harica (актуальные бандлы содержат и Harica, и GlobalSign — переход бесшовный).
- Не пинить сертификат и не указывать устаревший собственный ca-bundle — иначе после миграции соединение упадёт: certificate verify failed / unable to get local issuer certificate.
- Не отключать проверку сертификата (verify=false) как «обход» — это уязвимость (MITM), а не восстановление доверия. Правильно — обновить СА-бандл.
- Держать СА-бандл обновляемым: с автообновлением системного хранилища ОС это происходит само; при собственном бандле — обновлять его (детали для PHP — раздел 8).

4. Публичные функции

4.1. get_campaign_dict()

Справочник рекламных кампаний аккаунта.

HTTP: GET /api/client/campaign. Пагинации нет — весь список за один запрос.

Поле	Тип	Источник (ключ API)
campaign_id	string	id (integer в JSON → string)
campaign_name	string	title
account_id	integer	константа (значение 1 — пример, задаётся на стороне клиента)
source_type_id	integer	константа (значение 9 — пример, задаётся на стороне клиента)
product_id	integer	константа (значение 1 — пример, задаётся на стороне клиента)
product_name	string	константа (значение "prod_test" — пример, задаётся на стороне клиента)
camp_type	string	константа (значение "camp_test" — пример, задаётся на стороне клиента)
camp_category	string	константа (значение "cat_test" — пример, задаётся на стороне клиента)
id_key_camp	string	вычисляется: account_id + "_" + campaign_id (пример: "1_25725956")
owner_id	integer	константа (значение 1 — пример, задаётся на стороне клиента)

Константные поля и id_key_camp заполняются на стороне клиента — не из API. Конкретные значения приведены как пример и должны быть заменены на актуальные при интеграции.

4.2. get_campaigns_daily_stat(date_from, date_to)

Статистика по кампаниям (без охватов) по дням. Гранулярность: campaign_id × date.

HTTP: POST /api/client/statistics, groupBy=DATE, 1 день за запрос, батчи по 10. CSV — ad-level строки. Суммировать Показы/Клики/Расход по (date, campaign_id).

Поле	Тип	Источник (колонка CSV)
date	string YYYY-MM-DD	День (DD.MM.YYYY → конвертировать)
campaign_id	string	из контекста батча (не из CSV)
impressions	float	Показы
clicks	float	Клики
costs_nds	float	Расход, ₽, с НДС
costs_without_nds	float	вычисляется: costs_nds / 1.22 (НДС 22%)
ak	float	константа (значение 0.5 — агентская комиссия 50%)
costs_nds_ak	float	вычисляется: costs_nds * (1 + ak)
costs_without_nds_ak	float	вычисляется: costs_without_nds * (1 + ak)
account_id	integer	константа (значение 1 — пример, задаётся на стороне клиента)
source_type_id	integer	константа (значение 9 — пример, задаётся на стороне клиента)
id_key_camp	string	вычисляется: "1_" + campaign_id

4.3. get_ads_daily_stat(date_from, date_to)

Статистика по объявлениям (без охватов) по дням. Гранулярность: ad_id × date.

Тот же endpoint что 4.2. Агрегация не нужна — одна строка CSV = одна строка результата.

Поле	Тип	Источник (колонка CSV)
date	string YYYY-MM-DD	День
campaign_id	string	из контекста батча
ad_id	string	ID баннера
ad_name	string	Название
impressions	float	Показы
clicks	float	Клики
costs_nds	float	Расход, Р, с НДС
costs_without_nds	float	вычисляется: $\text{costs_nds} / 1.22$ (НДС 22%)
ak	float	константа (значение 0.5 — агентская комиссия 50%)
costs_nds_ak	float	вычисляется: $\text{costs_nds} * (1 + \text{ak})$
costs_without_nds_ak	float	вычисляется: $\text{costs_without_nds} * (1 + \text{ak})$
account_id	integer	константа (значение 1 — пример, задаётся на стороне клиента)
source_type_id	integer	константа (значение 9 — пример, задаётся на стороне клиента)
id_key_camp	string	вычисляется: "1_" + campaign_id
id_key_ad	string	вычисляется: id_key_camp + "_" + ad_id (пример: "1_25725956_602761")

4.4. get_reach_campaigns_daily_stat(global_start_date, date_from, date_to)

Охват кампаний накопительным итогом по дням. Гранулярность: campaign_id × date.

Для каждого дня D: POST /api/client/statistics с dateFrom=global_start_date, dateTo=D, groupBy=NO_GROUP_BY. Взять строку 'Всего' → колонка 'Охват'.

Поле	Тип	Источник
date	string YYYY-MM-DD	параметр D цикла
campaign_id	string	из контекста батча
reach	float	'Охват' из строки 'Всего' CSV
increment	float	$\text{reach}[D] - \text{reach}[D-1]$; для первого дня = reach
account_id	integer	константа (значение 1 — пример, задаётся на стороне клиента)
source_type_id	integer	константа (значение 9 — пример, задаётся на стороне клиента)
id_key_camp	string	вычисляется: "1_" + campaign_id

4.5. get_reach_ads_daily_stat(global_start_date, date_from, date_to)

Охват объявлений накопительным итогом по дням. Гранулярность: ad_id × date. Reach суммируется по всем платформам.

Тот же запрос и кэш что 4.4. Брать ad-level строки CSV (не 'Всего'), затем суммировать reach по платформам: groupby(date, campaign_id, ad_id).

Поле	Тип	Источник (колонка CSV)
date	string YYYY-MM-DD	параметр D цикла
campaign_id	string	из контекста батча
ad_id	string	ID баннера
ad_name	string	Название
reach	float	Охват, сумма по платформам
increment	float	$\text{reach}[D] - \text{reach}[D-1]$ по (campaign_id, ad_id)
account_id	integer	константа (значение 1 — пример, задаётся на стороне клиента)
source_type_id	integer	константа (значение 9 — пример, задаётся на стороне клиента)
id_key_camp	string	вычисляется: "1_" + campaign_id
id_key_ad	string	вычисляется: id_key_camp + "_" + ad_id (пример: "1_25725956_602761")

4.6. get_video_ads_daily_stat(date_from, date_to)

Видео-статистика по объявлениям по дням. Только кампании advObjectType == 'VIDEO_BANNER'. Тот же endpoint POST /api/client/statistics с groupBy=DATE.

Отличия от BANNER: колонка расхода 'Расход, Р' (без НДС), нет колонки 'Платформа', есть видео-метрики.

Поле	Тип	Источник (колонка CSV)
date	string YYYY-MM-DD	День
campaign_id	string	из контекста батча
ad_id	string	ID баннера
ad_name	string	Название
impressions	float	Показы
viewable_views	float	Видимые показы
clicks	float	Клики
quartile_25	float	Досмотры по квартилям 25%
quartile_50	float	Досмотры по квартилям 50%
quartile_75	float	Досмотры по квартилям 75%
quartile_100	float	Досмотры по квартилям 100%
views_with_sound	float	Просмотры со звуком
costs_nds	float	Расход, Р (без НДС)
costs_without_nds	float	вычисляется: $\text{costs_nds} / 1.22$ (НДС 22%)
ak	float	константа (значение 0.5 — агентская комиссия 50%)
costs_nds_ak	float	вычисляется: $\text{costs_nds} * (1 + \text{ak})$
costs_without_nds_ak	float	вычисляется: $\text{costs_without_nds} * (1 + \text{ak})$
account_id	integer	константа (значение 1 — пример, задаётся на стороне клиента)
source_type_id	integer	константа (значение 9 — пример, задаётся на стороне клиента)
id_key_camp	string	вычисляется: "1_" + campaign_id
id_key_ad	string	вычисляется: id_key_camp + "_" + ad_id (пример: "1_25725956_602761")

4.7. get_admin_audit(date_from, date_to) — сводный аудит (таблица admin_audit)

Собственного эндпоинта в API нет — агрегат поверх 4.2 (get_campaigns_daily_stat): к строкам статистики кампаний джойнится owner_id из справочника (4.1, по campaign_id), затем метрики суммируются по ключу date × account_id × source_type_id × owner_id. 9 колонок:

Поле	Тип	Источник
date	string YYYY-MM-DD	из статистики кампаний (4.2)
account_id	integer	константа (значение 1 — пример)
source_type_id	integer	константа (значение 9 — пример)
owner_id	integer	из справочника кампаний (4.1), join по campaign_id
impressions	float	сумма по группе
clicks	float	сумма по группе
costs_nds	float	сумма по группе (округление до 2 знаков)
costs_without_nds	float	сумма по группе (округление до 2 знаков)
chef_flag	integer	константа (значение 1 — дефолт)

Наследует поведение и лимиты 4.2 (статистика по дням, батчи по 10, общий кэш raw_*.csv). Пустая статистика → пустой результат с этими колонками.

5. Алгоритм сбора статистики

- GET /api/client/campaign → массив кампаний.
Для get_video_ads_daily_stat: фильтр advObjectType == "VIDEO_BANNER".
- Список дат [date_from..date_to] (YYYY-MM-DD).
- Разбить кампании на батчи по 10 (BATCH_SIZE = 10).
- Для каждой пары (day, batch):
 - Если для каждого campaign_id в батче есть raw_{date_from}_{date_to}_{cid}_{day}.csv → читать из кэша, пропустить b-f.
 - Выдержать >= 3 сек с последнего submit.
 - POST /api/client/statistics:
{ "campaigns": ["id1", ...], "dateFrom": day, "dateTo": day, "groupBy": "DATE" }

- UUID.
- d. Polling каждые 7 сек: GET /api/client/statistics/{UUID}
NOT_STARTED/IN_PROGRESS → продолжать.
OK → скачать. ERROR → warning + пропустить батч.
Макс. 40 попыток → таймаут, пропустить батч.
- e. GET /api/client/statistics/report?UUID={UUID}
text/csv → один файл. application/zip → ZipArchive.
- f. Кэш: raw_{date_from}_{date_to}_{cid}_{day}.csv.
- g. Парсить CSV → список строк.

5. Собрать все строки, вернуть с фиксированными колонками.

Для охват-функций (4.4, 4.5):

- groupBy=NO_GROUP_BY, dateFrom=global_start_date, dateTo=day.
- Кэш: reach_{global_start_date}_{cid}_{day}.csv.
- После сбора: вычислить increment через diff по campaign_id (или campaign_id+ad_id для 4.5).
- Первый день: increment = reach (fillna).

6. Примеры запросов и ответов API

6.1. Авторизация

POST https://api-performance.ozon.ru/api/client/token
Content-Type: application/json

```
{"client_id": "94252485-1777222314558@advertising.performance.ozon.ru",
  "client_secret": "<secret>", "grant_type": "client_credentials"}
```

Ответ: {"access_token": "eyJhbGc...", "expires_in": 1800, "token_type": "Bearer"}

6.2. Запрос отчёта (submit)

POST https://api-performance.ozon.ru/api/client/statistics
Authorization: Bearer eyJhbGc...
Content-Type: application/json

```
{"campaigns": ["24251481", "24296538"], "dateFrom": "2026-04-24", "dateTo": "2026-04-24", "groupBy": "DATE"}
```

Ответ: {"UUID": "12f4dc10-5e37-4b5e-aadd-f9176f224dac", "state": "NOT_STARTED", ...}

6.3. Опрос статуса

GET https://api-performance.ozon.ru/api/client/statistics/12f4dc10-5e37-4b5e-aadd-f9176f224dac
Authorization: Bearer eyJhbGc...

Ответ (в работе): {"state": "IN_PROGRESS", "UUID": "12f4dc10-..."}

Ответ (готов): {"state": "OK", "UUID": "12f4dc10-..."}

6.4. Скачивание отчёта — формат CSV

GET https://api-performance.ozon.ru/api/client/statistics/report?UUID=12f4dc10-...

1 кампания → Content-Type: text/csv

2+ кампаний → Content-Type: application/zip (файлы внутри: {campaign_id}.csv)

Пример CSV (BANNER, groupBy=DATE):

```
;Рекламная кампания № 24251481, Кампания, период 24.04.2026-24.04.2026
День;ID баннера;Название;Показы;Клики;CTR (%);Охват;...;Расход, ₽, с НДС
24.04.2026;602761;21.04-27.04_Порошок;86110;62;0,07;...;25833,00
24.04.2026;602764;21.04-27.04_Стирка;61550;45;0,07;...;18465,00
Всего;;...
```

Парсинг: найти строку-заголовок (начинается с "День;"),

skip строки "Всего" и "Корректировка".

7. Примеры таблиц на выходе

7.1. get_campaign_dict — справочник

campaign_id	campaign_name
25725956	Баннер 28.04.2026
24992642	igronik_5ka_ozon_ua_aon_reg_semya_cpm
24251481	Кампания_Апрель_2026

Константные и вычисляемые поля для тех же трёх строк:

campaign_id	account_id	source_type_id	product_id	product_name	camp_type	camp_category	id_key_camp	owner_id
25725956	1	9	1	prod_test	camp_test	cat_test	1_25725956	1
24992642	1	9	1	prod_test	camp_test	cat_test	1_24992642	1
24251481	1	9	1	prod_test	camp_test	cat_test	1_24251481	1

Значения константных полей приведены как пример — при интеграции заменить на актуальные.

7.2. get_campaigns_daily_stat — кампании × день

date	campaign_id	impressions	clicks	costs_nds
2026-04-24	24251481	147660	107	44298.0
2026-04-24	24296538	308342	166	92502.6
2026-04-25	24251481	147441	90	44232.3

Вычисляемые поля (значения для тех же 3 строк в том же порядке):

costs_without_nds	ak	costs_nds_ak	costs_without_nds_ak
36310.7	0.5	66447.0	54466.1
75821.0	0.5	138753.9	113731.5
36256.8	0.5	66348.5	54385.2

Константные поля account_id, source_type_id и вычисляемое id_key_camp (для тех же 3 строк):

account_id	source_type_id	id_key_camp
1	9	1_24251481
1	9	1_24296538
1	9	1_24251481

Значения account_id и source_type_id приведены как пример — при интеграции заменить на актуальные.

7.3. get_ads_daily_stat — объявления × день

date	campaign_id	ad_id	ad_name	impressions	clicks	costs_nds
2026-04-24	24296538	602634	Моб_Белый	145070	77	43521.0
2026-04-24	24296538	602637	Моб_Ржаной	163272	89	48981.6
2026-04-24	24251481	602761	Порошок	86110	62	25833.0

Вычисляемые поля (значения для тех же 3 строк в том же порядке):

costs_without_nds	ak	costs_nds_ak	costs_without_nds_ak
35673.0	0.5	65281.5	53509.5
40148.9	0.5	73472.4	60223.4
21174.6	0.5	38749.5	31761.9

Константные поля account_id, source_type_id и вычисляемые ключи id_key_camp, id_key_ad (для тех же 3 строк):

account_id	source_type_id	id_key_camp	id_key_ad
1	9	1_24296538	1_24296538_602634
1	9	1_24296538	1_24296538_602637
1	9	1_24251481	1_24251481_602761

Значения account_id и source_type_id приведены как пример — при интеграции заменить на актуальные.

7.4. get_reach_campaigns_daily_stat — охват кампаний × день

date	campaign_id	reach	increment
2026-04-24	24251481	622399	622399
2026-04-25	24251481	724007	101608
2026-04-24	24296538	1232180	1232180

Константные поля account_id, source_type_id и вычисляемое id_key_camp (для тех же 3 строк в том же порядке):

account_id	source_type_id	id_key_camp
1	9	1_24251481
1	9	1_24251481
1	9	1_24296538

Значения account_id и source_type_id приведены как пример — при интеграции заменить на актуальные.

7.5. get_reach_ads_daily_stat — охват объявлений × день

date	campaign_id	ad_id	ad_name	reach	increment
2026-04-24	24251481	598953	14.04–20.04_Сок	66758	66758
2026-04-25	24251481	598953	14.04–20.04_Сок	66758	0
2026-04-24	24251481	598956	14.04–20.04_Туалетная	87734	87734

Константные поля account_id, source_type_id и вычисляемые ключи id_key_camp, id_key_ad (для тех же 3 строк в том же порядке):

account_id	source_type_id	id_key_camp	id_key_ad
1	9	1_24251481	1_24251481_598953
1	9	1_24251481	1_24251481_598953
1	9	1_24251481	1_24251481_598956

Значения account_id и source_type_id приведены как пример — при интеграции заменить на актуальные.

7.6. get_video_ads_daily_stat — видео × объявление × день

date	campaign_id	ad_id	ad_name	impression	v_views	clicks
2025-08-05	16568770	401234	OLV_Волга	15420	12100	34

Видео-метрики (значения для той же строки):

q25	q50	q75	q100	views_with_sound	costs_nds
9800	7200	5100	3200	6800	18504.0

Вычисляемые поля (значения для той же строки):

costs_without_nds	ak	costs_nds_ak	costs_without_nds_ak
15167.2	0.5	27756.0	22750.8

Константные поля account_id, source_type_id и вычисляемые ключи id_key_camp, id_key_ad (для той же строки):

account_id	source_type_id	id_key_camp	id_key_ad
1	9	1_16568770	1_16568770_401234

Примечание: VIDEO_BANNER кампании работали в Aug 2025. Для Apr 2026 данных нет — возвращается пустой массив.

Значения account_id и source_type_id приведены как пример — при интеграции заменить на актуальные.

7.7. get_admin_audit — сводный аудит (таблица admin_audit)

date	account_i	source_type_id	owner_id	impressions	clicks	costs_nds	costs_without_nd	chef_flag
2026-04-24	1	9	1	456002	273	136801.0	112131.97	1
2026-04-25	1	9	1	455250	256	136575.0	111946.72	1

Все кампании аккаунта имеют owner_id = 1 → одна строка на день. Значения account_id, source_type_id, chef_flag приведены как пример.

8. Рекомендации по реализации на PHP

- PHP >= 8.1. Composer-проект.
- HTTP-клиент: guzzlehttp/guzzle с retry-middleware для 429 (exponential backoff).
- Кэширование токена: поля \$token, \$expiresAt в классе. ensureToken() перед каждым запросом: обновлять если time() >= \$expiresAt - 60.
- parseNum(\$v): (float) str_replace(',', '.', trim(str_replace(["\xc2\xa0", " ", "\t"], "", \$v))).
- parseDateStr(\$v): preg_match('/^(\d{2})\.(\d{2})\.(\d{4})\$/', ...) → YYYY-MM-DD; null если не совпало.
- TLS / CA-бандл: Guzzle/cURL по умолчанию проверяют сертификат сервера по системному CA-бандлу. С 2026 Ozon переходит на корневой УЦ Harica (с GlobalSign) — убедиться, что бандл актуален и содержит корень Harica. Не задавать устаревший CURLOPT_CAINFO / 'verify' => '/path/old/cacert.pem' и не отключать проверку ('verify' => false). При ошибке certificate verify failed — обновить бандл (composer require composer/ca-bundle или системные ca-certificates), а не отключать verify.
- Декодирование cp1251: mb_convert_encoding(\$csvContent, 'UTF-8', 'CP1251') перед парсингом.
- Поиск заголовка CSV: первая строка начинающаяся с 'День;'.
• ZIP: ZipArchive → читать каждый файл, привязать к campaign_id по имени файла.
- Логирование: PSR-3 / monolog. Все skip-предупреждения уровня warning.
- Тесты: моки HTTP на все 7 функций (включая агрегат get_admin_audit) + retry на 429 + таймаут polling.

Константы (не хардкодить в цикле)

Константа	Значение
POLL_INTERVAL_SEC	7
POLL_MAX_ATTEMPTS	40
TOKEN_REFRESH_LEEWAY_SEC	60
RATE_LIMIT_BASE_SEC	10
RATE_LIMIT_RETRY_MAX	5
MIN_SUBMIT_INTERVAL_SEC	3
MAX_CONCURRENT	3
BATCH_SIZE	10

9. Критерии приёмки

- Все 7 функций возвращают данные с теми же колонками и в том же порядке, что в Python-эталоне ozon_performance.py.
- get_admin_audit суммирует метрики по date × account_id × source_type_id × owner_id, проставляет chef_flag = 1, деньги округляет до 2 знаков.
- Корректная работа на реальных учётных данных: 88 кампаний, период > 7 дней, без 429.
- Токен переподписывается автоматически при многочасовом цикле.
- Лимит 3 параллельных UUID не превышает; между submit'ами >= 3 сек.
- Числа парсятся корректно (запятая→точка), CSV декодируется из cp1251, даты в ISO YYYY-MM-DD.
- При сбое любого батча процесс продолжает работать, ошибка попадает в лог.
- Для первого дня в охват-функциях: increment == reach.
- Строки 'Всего' и 'Корректировка' не попадают в результат.
- campaign_id и ad_id — всегда строки (не числа).
- Соединение с API использует актуальный CA-бандл (доверие к корню Harica); проверка TLS-сертификата не отключена.
- README с примерами вызова + .env.example.